

Hackathon (Non-Graded)

"The Digital Divide Dashboard"

Format: Solo or pairs, self-formed · **Duration:** ~3 hours · **Scoring:** None — this is a rehearsal, not a competition

Background

Two people can live in different countries and have completely different relationships with the internet — one streams 4K video on a phone, another shares one shared connection with a whole village. The World Bank tracks this every year through indicators like **internet penetration** and **mobile subscriptions**.

Today, you're a data journalist. Your job: turn a raw, messy World Bank-style export into a short visual story about the global digital divide — and ship it as a small interactive app.

This is a **practice run**. Nobody is graded. The goal is to go through the full motion — clean data, find a story, visualize it, wrap it in an app, demo it — once, in a low-pressure setting, before Day 2 raises the stakes.

The Dataset

Extract World bank data

```
countries = ["US", "GB", "DE", "JP", "KR", "FR", "BR", "CN", "MX", "ZA",  
            "AR", "ID", "IN", "VN", "EG", "NG", "KE", "ET"]
```

```
indicators = {  
    "IT.NET.USER.ZS": "internet_users_pct",  
    "IT.CEL.SETS.P2": "mobile_subscriptions_per_100",  
}
```

Filter below columns:

Column	Description
country	Country name (⚠️ inconsistently formatted — see below)
year	Year, 2005–2023 (⚠️ mixed types)
internet_users_pct	Internet users, % of population (⚠️ mixed types, a couple of impossible

Column	Description
	values)
mobile_subscriptions_per_100	Mobile subscriptions per 100 people (⚠ missing values, one impossible value)

Known data quality issues (find and fix all of these):

- The same country appears under multiple spellings/formats — e.g. you'll find "United Kingdom", "UK", "U.K.", and versions with stray whitespace or ALL CAPS all referring to the same country. There are 18 real countries hiding behind ~40 raw text variants.
- internet_users_pct is sometimes a number, sometimes a string like "36.6%".
- year is sometimes an integer, sometimes a string.
- Some rows have missing values in the numeric columns.
- A handful of rows are exact duplicates.
- A few values are physically impossible (e.g. a percentage over 100, or a negative subscription count) — data entry errors to catch, not just delete blindly.

Tasks

1. Clean it (pandas).

- Build a mapping to collapse country-name variants down to one canonical name per country (18 total).
- Fix data types (numbers should be numeric, years should be integers).
- Handle missing values sensibly — decide and justify your approach (drop vs. impute).
- Remove duplicate rows.
- Flag and handle the impossible outlier values.

2. Find a story (matplotlib).

- Plot internet adoption over time for 4–5 countries of your choice on one chart. With only 19 annual points per country, a single noisy year (or a cleaning miss you didn't catch) can dominate the line — smooth it with a **3-year rolling average** and plot it alongside the raw values:

```
df = df.sort_values(["country", "year"])
df["internet_pct_3yr_avg"] = (
    df.groupby("country")["internet_users_pct"]
    .transform(lambda x:
        x.rolling(3, min_periods=1).mean()))
```

(min_periods=1 keeps the first two years from turning into NaN — decide for yourself if that's the right call, or if you'd rather let them be blank.)

- Plot a bar chart of the top 5 and bottom 5 countries by most recent year's internet penetration — `df.sort_values("internet_users_pct", ascending=False)`.

3. Put it on a map (folium).

- A choropleth map of internet users % for the latest available year, one color per country.

4. Wrap it in an app (streamlit).

- A dropdown to pick a country → show its trend line and current stats.
- Embed the map somewhere on the page (folium renders fine inside Streamlit via `st.components.v1.html` or `streamlit-folium`).

Extra points for early finishers

Nobody's scored today, so these exist purely for extra practice — reach for them once your core app is working.

- **Sorting.** You already need to rank countries for the top-5/bottom-5 chart above — that's a sort on a value that already exists in the data. Stretch: sort by *growth* instead of *level*. Which countries gained the most internet-users percentage points between their first and last available year? That means computing a new value (last – first) and sorting on it, not just sorting an existing column.
- **Stack.** Internet adoption should only ever go up or hold steady over time — a year-over-year drop usually means either a real event (e.g. an outage) or a cleaning miss you didn't catch. Walk each country's cleaned values in order using a stack: push a year whenever the value rises above what's on top of the stack, and pop-and-flag whenever it drops. This is a simple monotonic-stack pattern, and it doubles as a sanity check on your own cleaning.
- **Queue.** You just built a 3-year rolling average with pandas' `.rolling()` above — try reimplementing it by hand with `collections.deque(maxlen=3)`: push each new year's value onto the deque, average whatever's currently in it, and let the deque automatically evict the oldest year once you go past 3. It's a good way to see what `.rolling()` is actually doing underneath.

Deliverable

- A 2-minute informal walkthrough to your table/mentor — no slides needed.
- Code shared via Google drive / GitHub

What "done" looks like

No scoring, but a working app that runs without errors, shows one clear insight, and reflects genuinely cleaned data (not just `dropna()` on everything) is the bar.